Министерство цифрового развития, связи и массовых коммуникаций Российской Федерации федеральное государственное бюджетное образовательное учреждение высшего образования
«Сибирский государственный университет телекоммуникаций и информатики»

Кафедра вычислительных систем

ОТЧЕТ
по практической работе 7

по дисциплине «**Основы разработки на Java**»

Выполнил:
студент гр. ИС-542
«11» октября 2026 г. _____ /Кузугашев С.В./

Проверил:
Ассистент кафедры ВС
«12» октября 2026 г. _____ /Гузенков С.К./

Оценка «_____»

Новосибирск 2026

Оглавление

Задание 3
Выполнение работы 5
Структура проекта 5
Задача 1. Каталог курсов в UTF-8 5
Задача 2. Buffered Reader/Writer и граница bytes → chars 6
Задача 3. Try-with-resources и порядок закрытия 7
Задача 4. Ленивые строки и владение Stream 8
Задача 5. Бинарный формат каталога 9
Задача 6. Временный файл и atomic move 10
Задача 7. Частично повреждённый импорт 11
Задача 8. Потоковый импорт большого файла 12
Задача 9. Версионируемый формат и миграция 14
Ошибки и их исправление 16
Заключение 17
Приложение 18

Задание

Дисциплина «Программирование». Лабораторная работа №7. Тема: работа с файлами и потоками ввода-вывода Java — кодировки, буферизация, try-with-resources, ленивые Stream, бинарные и версионируемые форматы, атомарная замена и потоковый импорт. Среда: OpenJDK 21.

Исходный код лабораторной работы опубликован в открытом репозитории: https://github.com/Forlein/SibGUTI-Java-Labs (папка WORK_Java/laboratory_work_7).

В работе реализованы 9 заданий:

1. Каталог курсов: сохранение и чтение в UTF-8, обработка отсутствующего файла и каталога вместо файла.

2. Текстовая обработка через BufferedReader/BufferedWriter со счётчиками строк и символов, объяснение границы bytes → chars.

3. Try-with-resources: pipeline с исключением в середине и доказательство правильного порядка close.

4. Files.lines только в try-with-resources; воспроизведение ошибочного возврата Stream наружу.

5. Бинарный формат: magic, version, count и записи через DataOutputStream/DataInputStream; отказ для неизвестной версии и обрезанного файла.

6. Временный файл в том же каталоге, flush/close и замена целевого файла через move с fallback без atomic move.

7. Импорт с ошибками и номерами строк: all-or-nothing, контракт результата зафиксирован sealed-типом.

8. Потоковый импорт большого файла: счётчики, ограниченный список ошибок, отмена после заданного порога.

9. Версионируемый формат: schema version, checksum, миграция v1 → v2, отказ будущей версии.

Работа выполнена на оценку «5»: реализованы все 9 заданий, каждое собрано и запущено, программы выдают корректный результат.

Выполнение работы

Ниже по каждой задаче приведён ключевой фрагмент кода (микро-листинг) и вывод программы; полные исходники лежат в папках 1–9 проекта.

Структура проекта

Каждое задание вынесено в отдельную папку с номером, внутри которой лежит файл Main.java. Классы с одинаковыми именами (Course, Main и т. п.) продублированы в папках, поэтому задания собираются и запускаются по отдельности. Сборка: javac -encoding UTF-8 -d out <папка>\Main.java; запуск: java -Dstdout.encoding=UTF-8 -cp out Main.

Папка	Файлы и задание
1	Main.java — каталог курсов в UTF-8
2	Main.java — BufferedReader/BufferedWriter
3	Main.java — try-with-resources
4	Main.java — Files.lines и владение Stream
5	Main.java — бинарный формат
6	Main.java — временный файл и atomic move
7	Main.java — частично повреждённый импорт
8	Main.java — потоковый импорт большого файла
9	Main.java — версионируемый формат
Корень	README.md, Отчет_ЛР7_Java.docx

Задача 1. Каталог курсов в UTF-8

В первой задаче каталог курсов сохраняется в текстовый файл в кодировке UTF-8 и читается обратно. Перед записью и чтением проверяется, не является ли путь каталогом, а при чтении отдельно обрабатывается отсутствие файла. Ошибка не проглатывается: программа печатает сообщение в stderr и завершается с кодом 1, поэтому ложного успеха не возникает.

Листинг – 1 Сохранение и чтение каталога с проверкой ошибок (1/Main.java)

```
1     try {
2         saveCatalog(catalog, courses);
3         List<Course> loaded = loadCatalog(catalog);
4         for (Course course : loaded) {
```

```
5             System.out.println(course.name() + " — " + course.hours() +
      " ч.");
6         }
7         System.out.println("Каталог сохранён и прочитан: " +
      loaded.size() + " курсов");
8     } catch (IOException | IllegalArgumentException e) {
9         System.err.println("Ошибка каталога курсов: " + e.getMessage());
10        System.exit(1);
11    }
12
13    if (Files.isDirectory(path)) {
14        throw new IOException("по пути " + path.toAbsolutePath()
15                + " находится каталог, а не файл");
16    }
17    if (!Files.exists(path)) {
18        throw new IOException("файл не найден: " +
      path.toAbsolutePath());
19    }
```

Вывод программы (1/Main.java)

```
1     Java. Базовый курс — 72 ч.
2     Алгоритмы и структуры данных — 108 ч.
3     Базы данных — 64 ч.
4     Веб-разработка — 96 ч.
5     Каталог сохранён и прочитан: 4 курсов
```

Программа сохраняет четыре курса в courses.txt в UTF-8, затем читает файл и печатает записи. Отсутствие файла и путь-каталог не игнорируются: выбрасывается IOException, сообщение выводится в stderr, процесс завершается с кодом 1.

Задача 2. Buffered Reader/Writer и граница bytes → chars

Во второй задаче текстовая обработка переписана через BufferedReader и BufferedWriter. InputStreamReader декодирует байты файла в UTF-16 char, а OutputStreamWriter кодирует char обратно в UTF-8 bytes; буферы только уменьшают число обращений к файлу. Программа считает строки, UTF-16 char и Unicode code points, поэтому видно, что эмодзи занимает две char.

Листинг – 2 Буферизованное чтение, запись и счётчики (2/Main.java)

```
1     try (BufferedReader reader = new BufferedReader(
2             new InputStreamReader(Files.newInputStream(input),
      StandardCharsets.UTF_8));
```

```
3          BufferedWriter writer = new BufferedWriter(
4                  new OutputStreamWriter(Files.newOutputStream(output),
    StandardCharsets.UTF_8))) {
5          String line;
6          while ((line = reader.readLine()) != null) {
7              lines++;
8              chars += line.length();
9              codePoints += line.codePointCount(0, line.length());
10             writer.write(lines + ": " + line);
11             writer.newLine();
12         }
13         writer.flush();
14     }
```

Вывод программы (2/Main.java)

```
1    Создан демонстрационный файл: input.txt
2    Строк: 3, UTF-16 char: 43, code points: 42
3    Граница bytes -> chars: InputStream отдаёт байты, InputStreamReader
     декодирует их в UTF-16 char; BufferedWriter кодирует char обратно в
     UTF-8 bytes.
```

BufferedReader.readLine() возвращает уже декодированные String, поэтому счётчик строк равен 3. UTF-16 char — 43, code points — 42: эмодзи 😀 занимает суррогатную пару. Граница bytes → chars находится в InputStreamReader, а обратная граница chars → bytes — в OutputStreamWriter.

Задача 3. Try-with-resources и порядок закрытия

В третьей задаче собран pipeline чтения и записи, внутри которого после flush искусственно выбрасывается IOException. Оба ресурса объявлены в try-with-resources, поэтому закрываются автоматически. Ресурсы обёрнуты в TrackedReader и TrackedWriter, которые записывают порядок close; дополнительно проверяются флаги закрытия.

Листинг – 3 Pipeline с исключением и трассировкой close (3/Main.java)

```
1    try (TrackedReader reader = new TrackedReader(
2                Files.newBufferedReader(input,
     StandardCharsets.UTF_8));
3         TrackedWriter writer = new TrackedWriter(
4                Files.newBufferedWriter(output,
     StandardCharsets.UTF_8))) {
5        String line = reader.readLine();
6        writer.write(line == null ? "" : line.toUpperCase(Locale.ROOT));
```

```
7         writer.newLine();
8         writer.flush();
9         throw new IOException("искусственный сбой в середине pipeline");
10    }
11
12    @Override
13    public void close() throws IOException {
14        writerClosed = true;
15        CLOSE_ORDER.add("writer.close");
16        delegate.close();
17    }
```

Вывод программы (3/Main.java)

```
1     Перехвачено: искусственный сбой в середине pipeline
2     Порядок close: [writer.close, reader.close]
3     reader закрыт: true, writer закрыт: true
4     Оба ресурса закрыты; порядок обратен порядку объявления.
```

Искусственное исключение возникает после flush, но до выхода из try. try-with-resources закрывает writer, затем reader; в логе видно writer.close → reader.close. Флаги подтверждают, что оба ресурса закрыты, а порядок обратен порядку объявления.

Задача 4. Ленивые строки и владение Stream

В четвёртой задаче показано, что Files.lines возвращает ленивый Stream, который владеет файловым дескриптором. Ошибочный метод возвращает Stream наружу: terminal operation count() не закрывает поток, и onClose остаётся невызванным. Корректный метод потребляет Stream внутри try-with-resources, поэтому дескриптор закрывается автоматически.

Листинг – 4 Ошибочный возврат Stream и корректный try-with-resources (4/Main.java)

```
1     private static Stream<String> brokenLines(Path path) throws
      IOException {
2         return Files.lines(path, StandardCharsets.UTF_8);
3     }
4
5     private static long countLines(Path path) throws IOException {
6         try (Stream<String> lines = Files.lines(path,
      StandardCharsets.UTF_8)) {
7             return lines.count();
```

```
8         }
9     }
10
11    AtomicBoolean closed = new AtomicBoolean(false);
12    Stream<String> leaked = brokenLines(brokenPath).onClose(() ->
      closed.set(true));
13    System.out.println("Прочитано строк: " + leaked.count());
14    System.out.println("onClose вызван после terminal operation: " +
      closed.get());
15    leaked.close();
```

Вывод программы (4/Main.java)

```
1     --- Ошибочный вариант: stream возвращён наружу ---
2     Прочитано строк: 3
3     onClose вызван после terminal operation: false
4     Удаление при открытом stream: удалён
5     onClose после leaked.close(): true
6     После leaked.close(): файл уже отсутствует
7     Владелец ресурса: тот, кто открыл Files.lines(); при возврате stream
      наружу закрывать обязан вызывающий.
8     --- Корректный вариант: try-with-resources ---
9     Корректно посчитано строк: 2
10    После корректного try-with-resources: удалён
```

После terminal operation count() onClose не вызван — поток остаётся открытым, хотя ОС разрешила удалить файл. Только leaked.close() переводит флаг в true. В корректном варианте Files.lines находится в try-with-resources, поэтому дескриптор закрывается автоматически.

Задача 5. Бинарный формат каталога

В пятой задаче каталог записывается в бинарный файл через DataOutputStream: magic header, version, количество записей и сами записи (writeUTF для названия и writeInt для часов). Чтение выполняется через DataInputStream. Неизвестная версия отклоняется до чтения записей, а обрезанный файл даёт EOFException, который преобразуется в понятное исключение.

Листинг – 5 Заголовок и записи бинарного формата (5/Main.java)

```
1     try (DataOutputStream out = new DataOutputStream(
2             new BufferedOutputStream(Files.newOutputStream(path)))) {
3         out.writeInt(MAGIC);
```

```
4           out.writeInt(VERSION);
5           out.writeInt(courses.size());
6           for (Course course : courses) {
7               out.writeUTF(course.name());
8               out.writeInt(course.hours());
9           }
10      }
11
12      int magic = in.readInt();
13      if (magic != MAGIC) {
14          throw new InvalidCatalogException("неверный magic header");
15      }
16      int version = in.readInt();
17      if (version != VERSION) {
18          throw new InvalidCatalogException("неизвестная версия: " +
        version);
19      }
20      int count = in.readInt();
21      if (count < 0 || count > MAX_RECORDS) {
22          throw new InvalidCatalogException("недопустимое количество
        записей: " + count);
23      }
```

Вывод программы (5/Main.java)

```
1       Прочитан каталог: [Course[name=Java. Базовый курс, hours=72],
        Course[name=Алгоритмы и структуры данных, hours=108],
        Course[name=Базы данных, hours=64]]
2       неизвестная версия отклонён: неизвестная версия: 99
3       обрезанный файл отклонён: файл обрезан
```

Валидный файл читается: magic и version совпали, count=3. Файл с version=99 отклонён до чтения записей; обрезанный файл даёт EOFException, который преобразован в InvalidCatalogException.

Задача 6. Временный файл и atomic move

В шестой задаче новый каталог сначала записывается во временный файл, созданный в том же каталоге, что и целевой. После flush и close выполняется move с REPLACE_EXISTING и ATOMIC_MOVE. Если файловая система не поддерживает atomic move, программа выполняет обычную замену и явно сообщает, что атомарность не гарантирована.

Листинг – 6 Временный файл и замена целевого файла (6/Main.java)

```
    Path temp = Files.createTempFile(directory, target.getFileName() +
    ".", ".tmp");
    boolean moved = false;
    try {
        try (BufferedWriter writer = Files.newBufferedWriter(temp,
    StandardCharsets.UTF_8)) {
            for (String line : lines) {
                writer.write(line);
                writer.newLine();
            }
            writer.flush();
        }
        moveReplacing(temp, target);
        moved = true;
    } finally {
        if (!moved) {
            Files.deleteIfExists(temp);
        }
    }

    try {
        Files.move(temp, target, StandardCopyOption.REPLACE_EXISTING,
                StandardCopyOption.ATOMIC_MOVE);
    } catch (AtomicMoveNotSupportedException e) {
        Files.move(temp, target, StandardCopyOption.REPLACE_EXISTING);
    }
```

Вывод программы (6/Main.java)

```
Замена выполнена atomic move.
Содержимое после замены:
Java. Базовый курс
Базы данных
```

Новый каталог записан во временный файл в том же каталоге, затем заменяет target через ATOMIC_MOVE. На NTFS atomic move поддерживается; при AtomicMoveNotSupportedException выполняется обычный move с предупреждением.

Задача 7. Частично повреждённый импорт

В седьмой задаче импортёр собирает ошибки с номерами строк и не публикует половину каталога. Выбран контракт all-or-nothing: результат импорта представлен sealed-типом ImportResult с вариантами ImportSuccess и

ImportFailure. При любой ошибке возвращается ImportFailure со списком ImportError, а хранилище не получает частично разобранный каталог.

Листинг – 7 Контракт результата импорта (7/Main.java)

```
1     private sealed interface ImportResult permits ImportSuccess,
      ImportFailure {
2     }
3
4     private record ImportSuccess(List<Course> courses) implements
      ImportResult {
5     }
6
7     private record ImportFailure(List<ImportError> errors) implements
      ImportResult {
8     }
9
10    private record ImportError(int lineNumber, String message) {
11    }
12
13    if (!errors.isEmpty()) {
14        return new ImportFailure(List.copyOf(errors));
15    }
16    return new ImportSuccess(List.copyOf(parsed));
```

Вывод программы (7/Main.java)

```
1     До импорта: [Course[name=Старый курс, hours=10]]
2     Импорт отклонён, ошибок: 2
3       строка 2: неверное число часов: сто
4       строка 3: пустое название
5     После отклонённого импорта: [Course[name=Старый курс, hours=10]]
6     Успешный импорт опубликован: [Course[name=Java, hours=72],
      Course[name=Базы данных, hours=64]]
```

Повреждённый импорт вернул ImportFailure с ошибками в строках 2 и 3. store не получил частичный каталог: опубликованный список остался прежним. Корректный файл вернул ImportSuccess и был опубликован целиком.

Задача 8. Потоковый импорт большого файла

В восьмой задаче большой файл читается построчно через BufferedReader, без readAllLines. Счётчики строк, валидных записей и ошибок имеют постоянный размер, а список ошибок ограничен. После достижения

заданного порога ошибок импорт отменяется, поэтому память не растёт вместе с размером файла.

Листинг – 8 Построчный импорт с ограниченными ошибками (8/Main.java)

```
1     try (BufferedReader reader = Files.newBufferedReader(path,
      StandardCharsets.UTF_8)) {
2         String line;
3         while ((line = reader.readLine()) != null) {
4             totalLines++;
5             String[] parts = line.split(";", -1);
6             if (parts.length != 2) {
7                 collector.add(new ImportError(totalLines, "ожидается 2
      поля"));
8             }
9             if (collector.totalErrors() >= cancelAfterErrors) {
10                cancelled = true;
11                break;
12            }
13        }
14    }
15    return new ImportSummary(totalLines, validRecords,
      collector.totalErrors(),
16            collector.storedErrors(), cancelled);
17
18    private void add(ImportError error) {
19        total++;
20        if (stored.size() < maxStored) {
21            stored.add(error);
22        }
23    }
```

Вывод программы (8/Main.java)

```
1     --- Небольшой файл: ошибок меньше порога ---
2     Обработано строк: 5000
3     Корректных записей: 4995
4     Всего ошибок: 5
5     Сохранено ошибок: 5
6       строка 1000: неверное число часов: ошибка
7       строка 2000: неверное число часов: ошибка
8       строка 3000: неверное число часов: ошибка
9       строка 4000: неверное число часов: ошибка
10      строка 5000: неверное число часов: ошибка
11    Импорт отменён: false
12    --- Большой файл: отмена по порогу ---
13    Обработано строк: 10000
```

```
Корректных записей: 9990
Всего ошибок: 10
Сохранено ошибок: 5
  строка 1000: неверное число часов: ошибка
  строка 2000: неверное число часов: ошибка
  строка 3000: неверное число часов: ошибка
  строка 4000: неверное число часов: ошибка
  строка 5000: неверное число часов: ошибка
Импорт отменён: true
Память: файл читается построчно, хранится не более 5 ошибок, валидные записи не накапливаются.
```

Малый файл обработан полностью: 5000 строк, 5 ошибок, все 5 сохранены. На большом файле после 10-й ошибки импорт остановлен: обработано 10000 строк, сохранено только 5 ошибок, валидные записи не накапливались.

Задача 9. Версионируемый формат и миграция

В девятой задаче формат получил schema version и checksum. Заголовок содержит magic, версию, длину payload и CRC32; checksum проверяется до разбора объектов, поэтому повреждение обнаруживается до публикации. Версия 1 мигрирует в версию 2 с вычислением credits, а неизвестная будущая версия не читается как старая.

Листинг – 9 Версия, checksum и миграция v1 → v2 (9/Main.java)

```
version = in.readInt();
if (version != VERSION_1 && version != VERSION_2) {
    throw new UnsupportedVersionException(version);
}
payloadLength = in.readInt();
expectedChecksum = in.readInt();
byte[] payload = in.readNBytes(payloadLength);
CRC32 crc = new CRC32();
crc.update(payload);
if ((int) crc.getValue() != expectedChecksum) {
    throw new CatalogFormatException("checksum не совпал: данные повреждены");
}
return parsePayload(version, payload);

int credits = version == VERSION_1 ? creditsFromHours(hours) : in.readInt();
```

Вывод программы (9/Main.java)

```
До публикации: [Course[name=Старый курс, hours=36, credits=1]]
v2 прочитан (версия 2): [Course[name=Java. Базовый курс, hours=72, credits=2], Course[name=Базы данных, hours=108, credits=3]]
v1 миграция -> v2 (версия 1): [Course[name=Java. Базовый курс, hours=72, credits=2], Course[name=Базы данных, hours=108, credits=3]]
будущая версия отклонён: неизвестная версия формата: 3
повреждённый файл отклонён: checksum не совпал: данные повреждены
Опубликованный каталог не изменён после повреждения.
```

v2 читается напрямую; v1 мигрирует в v2 с вычислением credits. Версия 3 отклонена до разбора payload. Повреждение последнего байта обнаружено по CRC32 до публикации, поэтому опубликованный каталог не изменился.

Ошибки и их исправление

Отсутствие файла и каталог вместо файла (задача 1). Files.exists() и Files.isDirectory() проверяются до чтения и записи; вместо молчаливого пропуска выбрасывается IOException, и программа завершается с ненулевым кодом.

Кодировка UTF-8 (задачи 1–2). Кодировка платформы может отличаться, поэтому во всех текстовых операциях явно указан StandardCharsets.UTF_8, а сборка и запуск выполняются с -encoding UTF-8 и -Dstdout.encoding=UTF-8.

Порядок закрытия ресурсов (задача 3). try-with-resources закрывает ресурсы в обратном порядке объявления; для pipeline это writer, затем reader. Ручной close в неправильном порядке мог бы потерять буферизованные данные.

Утечка файлового дескриптора (задача 4). Files.lines возвращает ленивый Stream, владеющий дескриптором. Возврат наружу без закрытия оставляет ресурс открытым; корректно потреблять поток внутри try-with-resources.

Неизвестная версия и обрезанный файл (задачи 5, 9). Версия проверяется до разбора записей, а EOF и несовпадение CRC32 преобразуются в понятные исключения; частично прочитанные объекты не публикуются.

Атомарная замена и частичный импорт (задачи 6–7). Временный файл создаётся в том же каталоге, чтобы move был атомарным; при отсутствии ATOMIC_MOVE выполняется fallback с предупреждением. Импорт all-or-nothing не публикует половину каталога.

Заключение

В ходе выполнения лабораторной работы №7 были изучены и применены на практике средства файлового ввода-вывода Java (OpenJDK 21): кодировки, буферизация, try-with-resources, ленивые Stream, бинарные и версионируемые форматы, атомарная замена и потоковая обработка больших файлов.

В задачах 1–3 отработаны текстовая обработка и управление ресурсами: явная граница bytes → chars через InputStreamReader/OutputStreamWriter, счётчики строк и символов, а также гарантированное закрытие reader и writer в обратном порядке.

В задачах 4–6 исследованы ленивые потоки и надёжная публикация файлов: доказано, что Files.lines требует try-with-resources, реализованы бинарный формат с magic/version/count и замена целевого файла через временный файл и atomic move.

В задачах 7–9 построены контракты импорта и версионирования: all-or-nothing результат с номерами строк, потоковый импорт с ограниченной памятью и отменой по порогу, schema version с CRC32 и миграцией v1 → v2.

Все 9 заданий размещены в отдельных папках, каждое задание компилируется и выполняется без ошибок. Работа выполнена в полном объёме.

Приложение

Листинг – 10 1/Main.java

```
1     /*
2     Задание 1. Сохраните каталог курсов в UTF-8, прочитайте, обработайте
      отсутствие файла и существующий каталог вместо файла. Не ловите
      ошибку только ради печати и продолжения с ложным успехом.
3     */
4     import java.io.IOException;
5     import java.nio.charset.StandardCharsets;
6     import java.nio.file.Files;
7     import java.nio.file.Path;
8     import java.nio.file.Paths;
9     import java.nio.file.StandardOpenOption;
10    import java.util.ArrayList;
11    import java.util.List;
12
13    public class Main {
14        private static final Path DEFAULT_CATALOG =
      Paths.get("courses.txt");
15
16        public static void main(String[] args) {
17            Path catalog = args.length > 0 ? Paths.get(args[0]) :
      DEFAULT_CATALOG;
18            List<Course> courses = List.of(
19                    new Course("Java. Базовый курс", 72),
20                    new Course("Алгоритмы и структуры данных", 108),
21                    new Course("Базы данных", 64),
22                    new Course("Веб-разработка", 96)
23            );
24
25            try {
26                saveCatalog(catalog, courses);
27                List<Course> loaded = loadCatalog(catalog);
28                for (Course course : loaded) {
29                    System.out.println(course.name() + " — " +
      course.hours() + " ч.");
30                }
31                System.out.println("Каталог сохранён и прочитан: " +
      loaded.size() + " курсов");
32            } catch (IOException | IllegalArgumentException e) {
33                System.err.println("Ошибка каталога курсов: " +
      e.getMessage());
34                System.exit(1);
35            }
36        }
37
```

```
38      private static void saveCatalog(Path path, List<Course> courses)
    throws IOException {
39          requireNotDirectory(path);
40          List<String> lines = new ArrayList<>();
41          for (Course course : courses) {
42              lines.add(course.name() + "\t" + course.hours());
43          }
44          Files.write(path, lines, StandardCharsets.UTF_8,
45                  StandardOpenOption.CREATE,
    StandardOpenOption.TRUNCATE_EXISTING, StandardOpenOption.WRITE);
46      }
47
48      private static List<Course> loadCatalog(Path path) throws
    IOException {
49          if (Files.isDirectory(path)) {
50              throw new IOException("по пути " + path.toAbsolutePath()
    + " находится каталог, а не файл");
51          }
52          if (!Files.exists(path)) {
53              throw new IOException("файл не найден: " +
    path.toAbsolutePath());
54          }
55          List<String> lines = Files.readAllLines(path,
    StandardCharsets.UTF_8);
56          List<Course> courses = new ArrayList<>();
57          for (int i = 0; i < lines.size(); i++) {
58              String line = lines.get(i);
59              if (line.isBlank()) {
60                  continue;
61              }
62              String[] parts = line.split("\t", -1);
63              if (parts.length != 2) {
64                  throw new IOException("строка " + (i + 1) + " имеет
    неверный формат");
65              }
66              try {
67                  courses.add(new Course(parts[0],
    Integer.parseInt(parts[1])));
68              } catch (NumberFormatException e) {
69                  throw new IOException("строка " + (i + 1) + "
    содержит неверное число часов", e);
70              }
71          }
72          return courses;
73      }
74
75      private static void requireNotDirectory(Path path) throws
    IOException {
76          if (Files.isDirectory(path)) {
```

```
77                throw new IOException("по пути " + path.toAbsolutePath()
    + " находится каталог, а не файл");
78            }
79        }
80
81        private record Course(String name, int hours) {
82            private Course {
83                if (name == null || name.isBlank()) {
84                    throw new IllegalArgumentException("название курса
    не должно быть пустым");
85                }
86                if (hours <= 0) {
87                    throw new IllegalArgumentException("часы курса
    должны быть положительными");
88                }
89            }
90        }
91    }
```

Листинг – 11 2/Main.java

```
1     /*
2     Задание 2. `Reader`/`Writer`
3
4     Перепишите текстовую обработку через buffered reader/writer,
      считайте строки и символы. Объясните границу bytes → chars.
5     */
6     import java.io.BufferedReader;
7     import java.io.BufferedWriter;
8     import java.io.IOException;
9     import java.io.InputStreamReader;
10    import java.io.OutputStreamWriter;
11    import java.nio.charset.StandardCharsets;
12    import java.nio.file.Files;
13    import java.nio.file.Path;
14    import java.nio.file.Paths;
15    import java.util.Locale;
16
17    public class Main {
18        public static void main(String[] args) {
19            Path input = args.length > 0 ? Paths.get(args[0]) :
      Paths.get("input.txt");
20            Path output = args.length > 1 ? Paths.get(args[1]) :
      Paths.get("output.txt");
21
22            try {
23                if (!Files.exists(input)) {
```

```
24                  Files.writeString(input, "Привет, мир!\nJava и
    Unicode: 😜\nТретья строка\n", StandardCharsets.UTF_8);
25                  System.out.println("Создан демонстрационный файл: "
    + input.toAbsolutePath());
26              }
27
28              Statistics statistics = process(input, output);
29              System.out.printf(Locale.ROOT, "Строк: %d, UTF-16 char:
    %d, code points: %d%n",
30                      statistics.lines(), statistics.chars(),
    statistics.codePoints());
31              System.out.println("Граница bytes -> chars: InputStream
    отдаёт байты, InputStreamReader декодирует их в UTF-16 char;
    BufferedWriter кодирует char обратно в UTF-8 bytes.");
32          } catch (IOException e) {
33              System.err.println("Ошибка обработки текста: " +
    e.getMessage());
34              System.exit(1);
35          }
36      }
37
38      private static Statistics process(Path input, Path output)
    throws IOException {
39          if
    (input.toAbsolutePath().normalize().equals(output.toAbsolutePath().n
    ormalize())) {
40              throw new IOException("входной и выходной файлы должны
    различаться");
41          }
42
43          long lines = 0;
44          long chars = 0;
45          long codePoints = 0;
46
47          try (BufferedReader reader = new BufferedReader(
48                  new InputStreamReader(Files.newInputStream(input),
    StandardCharsets.UTF_8));
49               BufferedWriter writer = new BufferedWriter(
50                       new
    OutputStreamWriter(Files.newOutputStream(output),
    StandardCharsets.UTF_8))) {
51
52              String line;
53              while ((line = reader.readLine()) != null) {
54                  lines++;
55                  chars += line.length();
56                  codePoints += line.codePointCount(0, line.length());
57                  writer.write(lines + ": " + line);
58                  writer.newLine();
```

```
59              }
60              writer.flush();
61          }
62
63          return new Statistics(lines, chars, codePoints);
64      }
65
66      private record Statistics(long lines, long chars, long
    codePoints) {
67      }
68  }
```

Листинг – 12 3/Main.java

```
1   /*
2   Задание 3. Try-with-resources
3
4   Создайте pipeline чтения и записи, в середине которого возникает
    исключение. Докажите закрытие обоих ресурсов и правильный порядок
    close.
5   */
6   import java.io.BufferedReader;
7   import java.io.BufferedWriter;
8   import java.io.Closeable;
9   import java.io.IOException;
10  import java.nio.charset.StandardCharsets;
11  import java.nio.file.Files;
12  import java.nio.file.Path;
13  import java.util.ArrayList;
14  import java.util.List;
15  import java.util.Locale;
16
17  public class Main {
18      private static final List<String> CLOSE_ORDER = new
    ArrayList<>();
19      private static boolean readerClosed;
20      private static boolean writerClosed;
21
22      public static void main(String[] args) {
23          boolean ok;
24          try {
25              ok = demonstrate();
26          } catch (IOException e) {
27              System.err.println("Ошибка демонстрации: " +
    e.getMessage());
28              ok = false;
29          }
30          if (!ok) {
```

```
31              System.exit(1);
32          }
33      }
34
35      private static boolean demonstrate() throws IOException {
36          Path dir = Files.createTempDirectory("try-with-resources-");
37          Path input = dir.resolve("input.txt");
38          Path output = dir.resolve("output.txt");
39
40          try {
41              Files.writeString(input, "данные для pipeline\n",
    StandardCharsets.UTF_8);
42
43              try {
44                  runPipeline(input, output);
45                  System.out.println("ОШИБКА: исключение не
    возникло");
46                  return false;
47              } catch (IOException e) {
48                  System.out.println("Перехвачено: " +
    e.getMessage());
49              }
50
51              System.out.println("Порядок close: " + CLOSE_ORDER);
52              System.out.println("reader закрыт: " + readerClosed + ",
    writer закрыт: " + writerClosed);
53
54              boolean correctOrder =
    CLOSE_ORDER.equals(List.of("writer.close", "reader.close"));
55              if (!readerClosed || !writerClosed || !correctOrder) {
56                  System.err.println("ОШИБКА: ресурсы закрыты
    неверно");
57                  return false;
58              }
59
60              System.out.println("Оба ресурса закрыты; порядок обратен
    порядку объявления.");
61              return true;
62          } finally {
63              Files.deleteIfExists(output);
64              Files.deleteIfExists(input);
65              Files.deleteIfExists(dir);
66          }
67      }
68
69      private static void runPipeline(Path input, Path output) throws
    IOException {
```

```
70          try (TrackedReader reader = new
    TrackedReader(Files.newBufferedReader(input,
    StandardCharsets.UTF_8));
71               TrackedWriter writer = new
    TrackedWriter(Files.newBufferedWriter(output,
    StandardCharsets.UTF_8))) {
72
73              String line = reader.readLine();
74              writer.write(line == null ? "" :
    line.toUpperCase(Locale.ROOT));
75              writer.newLine();
76              writer.flush();
77
78              throw new IOException("искусственный сбой в середине
    pipeline");
79          }
80      }
81
82      private static final class TrackedReader implements Closeable {
83          private final BufferedReader delegate;
84
85          private TrackedReader(BufferedReader delegate) {
86              this.delegate = delegate;
87          }
88
89          private String readLine() throws IOException {
90              return delegate.readLine();
91          }
92
93          @Override
94          public void close() throws IOException {
95              readerClosed = true;
96              CLOSE_ORDER.add("reader.close");
97              delegate.close();
98          }
99      }
100
101     private static final class TrackedWriter implements Closeable {
102         private final BufferedWriter delegate;
103
104         private TrackedWriter(BufferedWriter delegate) {
105             this.delegate = delegate;
106         }
107
108         private void write(String text) throws IOException {
109             delegate.write(text);
```

```
110          }
111
112          private void newLine() throws IOException {
113              delegate.newLine();
114          }
115
116          private void flush() throws IOException {
117              delegate.flush();
118          }
119
120          @Override
121          public void close() throws IOException {
122              writerClosed = true;
123              CLOSE_ORDER.add("writer.close");
124              delegate.close();
125          }
126      }
127  }
```

Листинг – 13 4/Main.java

```
1    /*
2    Задание 4. Ленивые строки
3
4    Используйте `Files.lines` только внутри try-with-resources.
     Воспроизведите ошибочный возврат stream наружу и объясните, кто
     обязан владеть ресурсом.
5    */
6    import java.io.IOException;
7    import java.nio.charset.StandardCharsets;
8    import java.nio.file.Files;
9    import java.nio.file.Path;
10   import java.util.concurrent.atomic.AtomicBoolean;
11   import java.util.stream.Stream;
12
13   public class Main {
14       public static void main(String[] args) {
15           Path brokenPath = null;
16           Path correctPath = null;
17           boolean ok = true;
18
19           try {
20               brokenPath = Files.createTempFile("lazy-lines-broken-",
     ".txt");
21               Files.writeString(brokenPath, "первая строка\nвторая
     строка\nтретья строка\n", StandardCharsets.UTF_8);
22
```

```
23              System.out.println("--- Ошибочный вариант: stream
    возвращён наружу ---");
24              AtomicBoolean closed = new AtomicBoolean(false);
25              Stream<String> leaked =
    brokenLines(brokenPath).onClose(() -> closed.set(true));
26              System.out.println("Прочитано строк: " +
    leaked.count());
27              System.out.println("onClose вызван после terminal
    operation: " + closed.get());
28              System.out.println("Удаление при открытом stream: " +
    deleteStatus(brokenPath));
29              leaked.close();
30              System.out.println("onClose после leaked.close(): " +
    closed.get());
31              System.out.println("После leaked.close(): " +
    deleteStatus(brokenPath));
32              System.out.println("Владелец ресурса: тот, кто открыл
    Files.lines(); при возврате stream наружу закрывать обязан
    вызывающий.");
33
34              correctPath = Files.createTempFile("lazy-lines-correct-
    ", ".txt");
35              Files.writeString(correctPath, "альфа\nбета\n",
    StandardCharsets.UTF_8);
36
37              System.out.println("--- Корректный вариант: try-with-
    resources ---");
38              long count = countLines(correctPath);
39              System.out.println("Корректно посчитано строк: " +
    count);
40              String status = deleteStatus(correctPath);
41              System.out.println("После корректного try-with-
    resources: " + status);
42              if (!status.equals("удалён")) {
43                  ok = false;
44              }
45          } catch (IOException e) {
46              System.err.println("Ошибка: " + e.getMessage());
47              ok = false;
48          } finally {
49              deleteQuietly(brokenPath);
50              deleteQuietly(correctPath);
51          }
52
53          if (!ok) {
54              System.exit(1);
55          }
56      }
57
```

```
58        private static Stream<String> brokenLines(Path path) throws
      IOException {
59            return Files.lines(path, StandardCharsets.UTF_8);
60        }
61
62        private static long countLines(Path path) throws IOException {
63            try (Stream<String> lines = Files.lines(path,
      StandardCharsets.UTF_8)) {
64                return lines.count();
65            }
66        }
67
68        private static String deleteStatus(Path path) {
69            try {
70                return Files.deleteIfExists(path) ? "удалён" : "файл уже
      отсутствует";
71            } catch (IOException e) {
72                return "не удалось (" + e.getClass().getSimpleName() +
      ") — дескриптор ещё открыт";
73            }
74        }
75
76        private static void deleteQuietly(Path path) {
77            if (path == null) {
78                return;
79            }
80            try {
81                Files.deleteIfExists(path);
82            } catch (IOException ignored) {
83            }
84        }
85    }
```

Листинг – 14 5/Main.java

```
1     /*
2     Задание 5. Бинарный формат
3
4     Запишите magic header, version, количество и записи через
      `DataOutputStream`; прочитайте через `DataInputStream`. Отклоняйте
      неизвестную версию и обрезанный файл.
5     */
6     import java.io.BufferedInputStream;
7     import java.io.BufferedOutputStream;
8     import java.io.DataInputStream;
9     import java.io.DataOutputStream;
10    import java.io.EOFException;
11    import java.io.IOException;
```

```
12  import java.io.RandomAccessFile;
13  import java.io.UTFDataFormatException;
14  import java.nio.file.Files;
15  import java.nio.file.Path;
16  import java.util.ArrayList;
17  import java.util.Comparator;
18  import java.util.List;
19  import java.util.stream.Stream;
20
21  public class Main {
22      private static final int MAGIC = 0x43415431;
23      private static final int VERSION = 1;
24      private static final int MAX_RECORDS = 10_000;
25
26      public static void main(String[] args) {
27          Path dir = null;
28          boolean ok = true;
29
30          try {
31              dir = Files.createTempDirectory("binary-catalog-");
32              Path valid = dir.resolve("valid.bin");
33              Path badVersion = dir.resolve("bad-version.bin");
34              Path truncated = dir.resolve("truncated.bin");
35
36              List<Course> courses = List.of(
37                      new Course("Java. Базовый курс", 72),
38                      new Course("Алгоритмы и структуры данных", 108),
39                      new Course("Базы данных", 64)
40              );
41
42              writeCatalog(valid, courses);
43              List<Course> loaded = readCatalog(valid);
44              System.out.println("Прочитан каталог: " + loaded);
45
46              writeRawHeader(badVersion, MAGIC, 99, 0);
47              ok &= expectRejected(badVersion, "неизвестная версия");
48
49              writeCatalog(truncated, courses);
50              try (RandomAccessFile file = new
    RandomAccessFile(truncated.toFile(), "rw")) {
51                  file.setLength(Math.max(0, file.length() - 2));
52              }
53              ok &= expectRejected(truncated, "обрезанный файл");
54          } catch (IOException e) {
55              System.err.println("Ошибка бинарного формата: " +
    e.getMessage());
```

```
56              ok = false;
57          } finally {
58              deleteRecursively(dir);
59          }
60
61          if (!ok) {
62              System.exit(1);
63          }
64      }
65
66      private static void writeCatalog(Path path, List<Course>
    courses) throws IOException {
67          try (DataOutputStream out = new DataOutputStream(
68                  new
    BufferedOutputStream(Files.newOutputStream(path)))) {
69              out.writeInt(MAGIC);
70              out.writeInt(VERSION);
71              out.writeInt(courses.size());
72              for (Course course : courses) {
73                  out.writeUTF(course.name());
74                  out.writeInt(course.hours());
75              }
76          }
77      }
78
79      private static List<Course> readCatalog(Path path) throws
    IOException {
80          try (DataInputStream in = new DataInputStream(
81                  new
    BufferedInputStream(Files.newInputStream(path)))) {
82              try {
83                  int magic = in.readInt();
84                  if (magic != MAGIC) {
85                      throw new InvalidCatalogException("неверный
    magic header");
86                  }
87
88                  int version = in.readInt();
89                  if (version != VERSION) {
90                      throw new InvalidCatalogException("неизвестная
    версия: " + version);
91                  }
92
93                  int count = in.readInt();
94                  if (count < 0 || count > MAX_RECORDS) {
95                      throw new InvalidCatalogException("недопустимое
    количество записей: " + count);
```

```
96                  }
97
98                  List<Course> courses = new ArrayList<>(count);
99                  for (int i = 0; i < count; i++) {
100                     String name = in.readUTF();
101                     int hours = in.readInt();
102                     if (name.isBlank()) {
103                         throw new InvalidCatalogException("пустое
    название в записи #" + (i + 1));
104                     }
105                     if (hours <= 0) {
106                         throw new InvalidCatalogException("неверные
    часы в записи #" + (i + 1));
107                     }
108                     courses.add(new Course(name, hours));
109                 }
110
111                 if (in.read() != -1) {
112                     throw new InvalidCatalogException("лишние данные
    после записей");
113                 }
114                 return courses;
115             } catch (EOFException e) {
116                 throw new InvalidCatalogException("файл обрезан",
    e);
117             } catch (UTFDataFormatException e) {
118                 throw new InvalidCatalogException("повреждённая
    строка в файле", e);
119             }
120         }
121     }
122
123     private static void writeRawHeader(Path path, int magic, int
    version, int count) throws IOException {
124         try (DataOutputStream out = new DataOutputStream(
125                 new
    BufferedOutputStream(Files.newOutputStream(path)))) {
126             out.writeInt(magic);
127             out.writeInt(version);
128             out.writeInt(count);
129         }
130     }
131
132     private static boolean expectRejected(Path path, String label) {
133         try {
134             List<Course> courses = readCatalog(path);
135             System.out.println("ОШИБКА: " + label + " был принят: "
    + courses);
```

```
136             return false;
137         } catch (InvalidCatalogException e) {
138             System.out.println(label + " отклонён: " +
    e.getMessage());
139             return true;
140         } catch (IOException e) {
141             System.out.println(label + " отклонён (I/O): " +
    e.getMessage());
142             return true;
143         }
144     }
145
146     private static void deleteRecursively(Path dir) {
147         if (dir == null) {
148             return;
149         }
150         try (Stream<Path> paths = Files.walk(dir)) {
151             paths.sorted(Comparator.reverseOrder()).forEach(path ->
    {
152                 try {
153                     Files.deleteIfExists(path);
154                 } catch (IOException ignored) {
155                 }
156             });
157         } catch (IOException ignored) {
158         }
159     }
160
161     private record Course(String name, int hours) {
162     }
163
164     private static final class InvalidCatalogException extends
    IOException {
165         private static final long serialVersionUID = 1L;
166
167         private InvalidCatalogException(String message) {
168             super(message);
169         }
170
171         private InvalidCatalogException(String message, Throwable
    cause) {
172             super(message, cause);
173         }
174     }
175 }
```

Листинг – 15 6/Main.java

```
1     /*
2     Задание 6. Временный файл и move
3
4     Записывайте новый каталог во временный файл того же каталога,
      flush/close, затем заменяйте целевой файл move-операцией.
      Обработайте отсутствие atomic move на файловой системе.
5     */
6     import java.io.BufferedWriter;
7     import java.io.IOException;
8     import java.nio.charset.StandardCharsets;
9     import java.nio.file.AtomicMoveNotSupportedException;
10    import java.nio.file.Files;
11    import java.nio.file.Path;
12    import java.nio.file.StandardCopyOption;
13    import java.util.Comparator;
14    import java.util.List;
15    import java.util.stream.Stream;
16
17    public class Main {
18        public static void main(String[] args) {
19            Path dir = null;
20            boolean ok = true;
21
22            try {
23                dir = Files.createTempDirectory("atomic-replace-");
24                Path target = dir.resolve("courses.txt");
25                Files.writeString(target, "старый каталог\n",
      StandardCharsets.UTF_8);
26
27                replaceCatalog(target, List.of("Java. Базовый курс",
      "Базы данных"));
28
29                String result = Files.readString(target,
      StandardCharsets.UTF_8);
30                System.out.println("Содержимое после замены:");
31                System.out.print(result);
32                String normalized = result.replace("\r\n",
      "\n").replace('\r', '\n');
33                if (!normalized.equals("Java. Базовый курс\nБазы
      данных\n")) {
34                    System.out.println("ОШИБКА: содержимое не
      совпадает");
35                    ok = false;
36                }
37            } catch (IOException e) {
38                System.err.println("Ошибка замены каталога: " +
      e.getMessage());
```

```
39              ok = false;
40          } finally {
41              deleteRecursively(dir);
42          }
43
44          if (!ok) {
45              System.exit(1);
46          }
47      }
48
49      private static void replaceCatalog(Path target, List<String>
    lines) throws IOException {
50          Path directory = target.toAbsolutePath().getParent();
51          Path temp = Files.createTempFile(directory,
    target.getFileName() + ".", ".tmp");
52          boolean moved = false;
53
54          try {
55              try (BufferedWriter writer =
    Files.newBufferedWriter(temp, StandardCharsets.UTF_8)) {
56                  for (String line : lines) {
57                      writer.write(line);
58                      writer.newLine();
59                  }
60                  writer.flush();
61              }
62
63              moveReplacing(temp, target);
64              moved = true;
65          } finally {
66              if (!moved) {
67                  Files.deleteIfExists(temp);
68              }
69          }
70      }
71
72      private static void moveReplacing(Path temp, Path target) throws
    IOException {
73          try {
74              Files.move(temp, target,
    StandardCopyOption.REPLACE_EXISTING,
    StandardCopyOption.ATOMIC_MOVE);
75              System.out.println("Замена выполнена atomic move.");
76          } catch (AtomicMoveNotSupportedException e) {
77              Files.move(temp, target,
    StandardCopyOption.REPLACE_EXISTING);
```

```
78                System.out.println("Atomic move не поддерживается:
    выполнена обычная замена, атомарность не гарантирована.");
79            }
80        }
81
82        private static void deleteRecursively(Path dir) {
83            if (dir == null) {
84                return;
85            }
86            try (Stream<Path> paths = Files.walk(dir)) {
87                paths.sorted(Comparator.reverseOrder()).forEach(path ->
    {
88                    try {
89                        Files.deleteIfExists(path);
90                    } catch (IOException ignored) {
91                    }
92                });
93            } catch (IOException ignored) {
94            }
95        }
96    }
```

Листинг – 16 7/Main.java

```
1     /*
2     Задание 7. Частично повреждённый импорт
3
4     Импортёр собирает ошибки с номерами строк и не публикует половину
      каталога. Выберите atomic all-or-nothing либо partial result и
      зафиксируйте контракт типом результата.
5     */
6     import java.io.BufferedReader;
7     import java.io.IOException;
8     import java.nio.charset.StandardCharsets;
9     import java.nio.file.Files;
10    import java.nio.file.Path;
11    import java.util.ArrayList;
12    import java.util.Comparator;
13    import java.util.List;
14    import java.util.stream.Stream;
15
16    public class Main {
17        public static void main(String[] args) {
18            Path dir = null;
19            boolean ok = true;
20
21            try {
22                dir = Files.createTempDirectory("import-");
```

```
23              Path bad = dir.resolve("bad.csv");
24              Path good = dir.resolve("good.csv");
25
26              Files.writeString(bad,
    "Java;72\nАлгоритмы;сто\n;10\nБазы данных;64\n",
    StandardCharsets.UTF_8);
27              Files.writeString(good, "Java;72\nБазы данных;64\n",
    StandardCharsets.UTF_8);
28
29              CatalogStore store = new CatalogStore(List.of(new
    Course("Старый курс", 10)));
30              System.out.println("До импорта: " + store.catalog());
31
32              ImportResult badResult = importCatalog(bad);
33              switch (badResult) {
34                  case ImportSuccess success -> {
35                      store.publish(success.courses());
36                      System.out.println("ОШИБКА: повреждённый импорт
    опубликован");
37                      ok = false;
38                  }
39                  case ImportFailure failure -> {
40                      System.out.println("Импорт отклонён, ошибок: " +
    failure.errors().size());
41                      for (ImportError error : failure.errors()) {
42                          System.out.println("  строка " +
    error.lineNumber() + ": " + error.message());
43                      }
44                  }
45              }
46
47              List<Course> expectedOld = List.of(new Course("Старый
    курс", 10));
48              System.out.println("После отклонённого импорта: " +
    store.catalog());
49              if (!store.catalog().equals(expectedOld)) {
50                  System.out.println("ОШИБКА: опубликована часть
    каталога");
51                  ok = false;
52              }
53
54              ImportResult goodResult = importCatalog(good);
55              switch (goodResult) {
56                  case ImportSuccess success -> {
57                      store.publish(success.courses());
58                      System.out.println("Успешный импорт опубликован:
    " + store.catalog());
59                  }
```

```
60                  case ImportFailure failure -> {
61                      System.out.println("ОШИБКА: корректный файл
    отклонён: " + failure.errors());
62                      ok = false;
63                  }
64              }
65          } catch (IOException e) {
66              System.err.println("Ошибка импорта: " + e.getMessage());
67              ok = false;
68          } finally {
69              deleteRecursively(dir);
70          }
71
72          if (!ok) {
73              System.exit(1);
74          }
75      }
76
77      private static ImportResult importCatalog(Path path) throws
    IOException {
78          List<Course> parsed = new ArrayList<>();
79          List<ImportError> errors = new ArrayList<>();
80
81          try (BufferedReader reader = Files.newBufferedReader(path,
    StandardCharsets.UTF_8)) {
82              String line;
83              int lineNumber = 0;
84
85              while ((line = reader.readLine()) != null) {
86                  lineNumber++;
87                  if (line.isBlank()) {
88                      continue;
89                  }
90
91                  String[] parts = line.split(";", -1);
92                  if (parts.length != 2) {
93                      errors.add(new ImportError(lineNumber,
    "ожидается формат название;часы"));
94                      continue;
95                  }
96
97                  String name = parts[0].trim();
98                  String hoursText = parts[1].trim();
99
100                 if (name.isEmpty()) {
```

```
101                     errors.add(new ImportError(lineNumber, "пустое
    название"));
102                     continue;
103                 }
104
105                 int hours;
106                 try {
107                     hours = Integer.parseInt(hoursText);
108                 } catch (NumberFormatException e) {
109                     errors.add(new ImportError(lineNumber, "неверное
    число часов: " + hoursText));
110                     continue;
111                 }
112
113                 if (hours <= 0) {
114                     errors.add(new ImportError(lineNumber, "часы
    должны быть положительными"));
115                     continue;
116                 }
117
118                 parsed.add(new Course(name, hours));
119             }
120         }
121
122         if (!errors.isEmpty()) {
123             return new ImportFailure(List.copyOf(errors));
124         }
125         return new ImportSuccess(List.copyOf(parsed));
126     }
127
128     private static void deleteRecursively(Path dir) {
129         if (dir == null) {
130             return;
131         }
132         try (Stream<Path> paths = Files.walk(dir)) {
133             paths.sorted(Comparator.reverseOrder()).forEach(path ->
    {
134                 try {
135                     Files.deleteIfExists(path);
136                 } catch (IOException ignored) {
137                 }
138             });
139         } catch (IOException ignored) {
140         }
141     }
142
```

```
143      private sealed interface ImportResult permits ImportSuccess,
     ImportFailure {
144      }
145
146      private record ImportSuccess(List<Course> courses) implements
     ImportResult {
147      }
148
149      private record ImportFailure(List<ImportError> errors)
     implements ImportResult {
150      }
151
152      private record ImportError(int lineNumber, String message) {
153      }
154
155      private record Course(String name, int hours) {
156      }
157
158      private static final class CatalogStore {
159          private List<Course> catalog;
160
161          private CatalogStore(List<Course> initial) {
162              this.catalog = List.copyOf(initial);
163          }
164
165          private void publish(List<Course> newCatalog) {
166              this.catalog = List.copyOf(newCatalog);
167          }
168
169          private List<Course> catalog() {
170              return catalog;
171          }
172      }
173  }
```

Листинг – 17 8/Main.java

```
1    /*
2    Задание 8. Потоковый импорт большого файла
3
4    Обработайте файл, превосходящий доступную память, без
     `readAllLines`. Поддержите счётчики, ограниченный список ошибок и
     отмену после заданного порога.
5    */
6    import java.io.BufferedReader;
7    import java.io.BufferedWriter;
```

```
8     import java.io.IOException;
9     import java.nio.charset.StandardCharsets;
10    import java.nio.file.Files;
11    import java.nio.file.Path;
12    import java.util.ArrayList;
13    import java.util.List;
14
15    public class Main {
16        private static final int MAX_STORED_ERRORS = 5;
17        private static final int CANCEL_AFTER_ERRORS = 10;
18
19        public static void main(String[] args) {
20            Path smallFile = null;
21            Path largeFile = null;
22            boolean ok = true;
23
24            try {
25                Path dir = Files.createTempDirectory("stream-import-");
26                smallFile = dir.resolve("small.csv");
27                largeFile = dir.resolve("large.csv");
28
29                writeSample(smallFile, 5_000, 1_000);
30                writeSample(largeFile, 50_000, 1_000);
31
32                System.out.println("--- Небольшой файл: ошибок меньше
      порога ---");
33                ImportSummary completed = importCatalog(smallFile,
      MAX_STORED_ERRORS, CANCEL_AFTER_ERRORS);
34                printSummary(completed);
35                if (completed.cancelled()) {
36                    System.out.println("ОШИБКА: импорт не должен был
      отменяться");
37                    ok = false;
38                }
39                if (completed.totalErrors() != 5 ||
      completed.sampleErrors().size() != 5) {
40                    System.out.println("ОШИБКА: неверные счётчики
      ошибок");
41                    ok = false;
42                }
43
44                System.out.println("--- Большой файл: отмена по порогу -
      --");
45                ImportSummary cancelled = importCatalog(largeFile,
      MAX_STORED_ERRORS, CANCEL_AFTER_ERRORS);
46                printSummary(cancelled);
47                if (!cancelled.cancelled()) {
```

```
48                  System.out.println("ОШИБКА: ожидалась отмена после
    порога");
49                  ok = false;
50              }
51              if (cancelled.totalErrors() != CANCEL_AFTER_ERRORS) {
52                  System.out.println("ОШИБКА: порог отмены не
    совпал");
53                  ok = false;
54              }
55              if (cancelled.sampleErrors().size() !=
    MAX_STORED_ERRORS) {
56                  System.out.println("ОШИБКА: список ошибок не
    ограничен");
57                  ok = false;
58              }
59
60              System.out.println("Память: файл читается построчно,
    хранится не более "
61                      + MAX_STORED_ERRORS + " ошибок, валидные записи
    не накапливаются.");
62          } catch (IOException e) {
63              System.err.println("Ошибка потокового импорта: " +
    e.getMessage());
64              ok = false;
65          } finally {
66              deleteQuietly(smallFile);
67              deleteQuietly(largeFile);
68              if (smallFile != null) {
69                  deleteQuietly(smallFile.getParent());
70              }
71          }
72
73          if (!ok) {
74              System.exit(1);
75          }
76      }
77
78      private static ImportSummary importCatalog(Path path, int
    maxStoredErrors, int cancelAfterErrors)
79              throws IOException {
80          if (maxStoredErrors < 0) {
81              throw new IllegalArgumentException("maxStoredErrors не
    может быть отрицательным");
82          }
83          if (cancelAfterErrors <= 0) {
84              throw new IllegalArgumentException("cancelAfterErrors
    должен быть положительным");
85          }
```

```
86
87          long totalLines = 0;
88          long validRecords = 0;
89          BoundedErrorCollector collector = new
    BoundedErrorCollector(maxStoredErrors);
90          boolean cancelled = false;
91
92          try (BufferedReader reader = Files.newBufferedReader(path,
    StandardCharsets.UTF_8)) {
93              String line;
94              while ((line = reader.readLine()) != null) {
95                  totalLines++;
96
97                  String[] parts = line.split(";", -1);
98                  if (parts.length != 2) {
99                      collector.add(new ImportError(totalLines,
    "ожидается 2 поля"));
100                 } else {
101                     String name = parts[0].trim();
102                     String hoursText = parts[1].trim();
103
104                     if (name.isEmpty()) {
105                         collector.add(new ImportError(totalLines,
    "пустое название"));
106                     } else {
107                         try {
108                             int hours = Integer.parseInt(hoursText);
109                             if (hours <= 0) {
110                                 collector.add(new
    ImportError(totalLines, "часы должны быть положительными"));
111                             } else {
112                                 validRecords++;
113                             }
114                         } catch (NumberFormatException e) {
115                             collector.add(new
    ImportError(totalLines, "неверное число часов: " + hoursText));
116                         }
117                     }
118                 }
119
120                 if (collector.totalErrors() >= cancelAfterErrors) {
121                     cancelled = true;
122                     break;
123                 }
124             }
125         }
126
```

```
127         return new ImportSummary(totalLines, validRecords,
    collector.totalErrors(),
128                 collector.storedErrors(), cancelled);
129     }
130
131     private static void writeSample(Path path, int totalLines, int
    errorEvery) throws IOException {
132         try (BufferedWriter writer = Files.newBufferedWriter(path,
    StandardCharsets.UTF_8)) {
133             for (int i = 1; i <= totalLines; i++) {
134                 if (i % errorEvery == 0) {
135                     writer.write("Курс " + i + ";ошибка");
136                 } else {
137                     writer.write("Курс " + i + ";" + (36 + (i % 5) *
    12));
138                 }
139                 writer.newLine();
140             }
141         }
142     }
143
144     private static void printSummary(ImportSummary summary) {
145         System.out.println("Обработано строк: " +
    summary.totalLines());
146         System.out.println("Корректных записей: " +
    summary.validRecords());
147         System.out.println("Всего ошибок: " +
    summary.totalErrors());
148         System.out.println("Сохранено ошибок: " +
    summary.sampleErrors().size());
149         for (ImportError error : summary.sampleErrors()) {
150             System.out.println("  строка " + error.lineNumber() + ":
    " + error.message());
151         }
152         System.out.println("Импорт отменён: " +
    summary.cancelled());
153     }
154
155     private static void deleteQuietly(Path path) {
156         if (path == null) {
157             return;
158         }
159         try {
160             Files.deleteIfExists(path);
161         } catch (IOException ignored) {
162         }
163     }
164
```

```
165      private record ImportSummary(long totalLines, long validRecords,
     long totalErrors,
166                                   List<ImportError> sampleErrors,
     boolean cancelled) {
167      }
168
169      private record ImportError(long lineNumber, String message) {
170      }
171
172      private static final class BoundedErrorCollector {
173          private final int maxStored;
174          private final List<ImportError> stored = new ArrayList<>();
175          private long total;
176
177          private BoundedErrorCollector(int maxStored) {
178              this.maxStored = maxStored;
179          }
180
181          private void add(ImportError error) {
182              total++;
183              if (stored.size() < maxStored) {
184                  stored.add(error);
185              }
186          }
187
188          private long totalErrors() {
189              return total;
190          }
191
192          private List<ImportError> storedErrors() {
193              return List.copyOf(stored);
194          }
195      }
196  }
```

Листинг – 18 9/Main.java

```
1     /*
2     Задание 9. Версионируемый формат
3
4     Добавьте schema version, checksum и migration из версии 1 в 2.
      Неизвестная будущая версия не читается как старая. Повреждение
      обнаруживается до публикации объектов.
5     */
6     import java.io.BufferedInputStream;
7     import java.io.BufferedOutputStream;
8     import java.io.ByteArrayInputStream;
```

```
9     import java.io.ByteArrayOutputStream;
10    import java.io.DataInputStream;
11    import java.io.DataOutputStream;
12    import java.io.EOFException;
13    import java.io.IOException;
14    import java.io.UTFDataFormatException;
15    import java.nio.file.Files;
16    import java.nio.file.Path;
17    import java.util.ArrayList;
18    import java.util.Comparator;
19    import java.util.List;
20    import java.util.stream.Stream;
21    import java.util.zip.CRC32;
22
23    public class Main {
24        private static final int MAGIC = 0x4341544C;
25        private static final int VERSION_1 = 1;
26        private static final int VERSION_2 = 2;
27        private static final int MAX_PAYLOAD = 64 * 1024 * 1024;
28        private static final int MAX_RECORDS = 100_000;
29
30        public static void main(String[] args) {
31            Path dir = null;
32            boolean ok = true;
33
34            try {
35                dir = Files.createTempDirectory("versioned-format-");
36                List<Course> source = List.of(
37                        new Course("Java. Базовый курс", 72, 2),
38                        new Course("Базы данных", 108, 3)
39                );
40
41                Path v2 = dir.resolve("catalog-v2.bin");
42                writeCatalog(v2, VERSION_2, source);
43
44                Publisher publisher = new Publisher(List.of(new
    Course("Старый курс", 36, 1)));
45                System.out.println("До публикации: " +
    publisher.catalog());
46
47                LoadedCatalog loadedV2 = readCatalog(v2);
48                publisher.publish(loadedV2.courses());
49                System.out.println("v2 прочитан (версия " +
    loadedV2.sourceVersion() + "): " + publisher.catalog());
50
51                Path v1 = dir.resolve("catalog-v1.bin");
```

```
52              writeCatalog(v1, VERSION_1, source);
53              LoadedCatalog loadedV1 = readCatalog(v1);
54              System.out.println("v1 миграция -> v2 (версия " +
    loadedV1.sourceVersion() + "): " + loadedV1.courses());
55
56              Path future = dir.resolve("catalog-v3.bin");
57              writeRawVersion(future, 3);
58              ok &= expectRejected(future, "будущая версия");
59
60              Path corrupted = dir.resolve("catalog-corrupted.bin");
61              byte[] bytes = Files.readAllBytes(v2);
62              bytes[bytes.length - 1] ^= 0x5A;
63              Files.write(corrupted, bytes);
64
65              List<Course> before = publisher.catalog();
66              ok &= expectRejected(corrupted, "повреждённый файл");
67              if (!publisher.catalog().equals(before)) {
68                  System.out.println("ОШИБКА: повреждённый файл
    изменил опубликованный каталог");
69                  ok = false;
70              } else {
71                  System.out.println("Опубликованный каталог не
    изменён после повреждения.");
72              }
73          } catch (IOException e) {
74              System.err.println("Ошибка версионируемого формата: " +
    e.getMessage());
75              ok = false;
76          } finally {
77              deleteRecursively(dir);
78          }
79
80          if (!ok) {
81              System.exit(1);
82          }
83      }
84
85      private static void writeCatalog(Path path, int version,
    List<Course> courses) throws IOException {
86          if (version != VERSION_1 && version != VERSION_2) {
87              throw new IllegalArgumentException("неподдерживаемая
    версия для записи: " + version);
88          }
89
90          ByteArrayOutputStream payloadBytes = new
    ByteArrayOutputStream();
```

```
91          try (DataOutputStream payload = new
    DataOutputStream(payloadBytes)) {
92              payload.writeInt(courses.size());
93              for (Course course : courses) {
94                  payload.writeUTF(course.name());
95                  payload.writeInt(course.hours());
96                  if (version == VERSION_2) {
97                      payload.writeInt(course.credits());
98                  }
99              }
100         }
101
102         byte[] payload = payloadBytes.toByteArray();
103         CRC32 crc = new CRC32();
104         crc.update(payload);
105
106         try (DataOutputStream out = new DataOutputStream(
107                 new
    BufferedOutputStream(Files.newOutputStream(path)))) {
108             out.writeInt(MAGIC);
109             out.writeInt(version);
110             out.writeInt(payload.length);
111             out.writeInt((int) crc.getValue());
112             out.write(payload);
113         }
114     }
115
116     private static LoadedCatalog readCatalog(Path path) throws
    IOException {
117         try (DataInputStream in = new DataInputStream(
118                 new
    BufferedInputStream(Files.newInputStream(path)))) {
119
120             int magic;
121             int version;
122             int payloadLength;
123             int expectedChecksum;
124
125             try {
126                 magic = in.readInt();
127                 if (magic != MAGIC) {
128                     throw new CatalogFormatException("неверный magic
    header");
129                 }
130
131                 version = in.readInt();
132                 if (version != VERSION_1 && version != VERSION_2) {
```

```
133                     throw new UnsupportedVersionException(version);
134                 }
135 
136                 payloadLength = in.readInt();
137                 if (payloadLength < 0 || payloadLength > 
    MAX_PAYLOAD) {
138                     throw new CatalogFormatException("недопустимая 
    длина payload: " + payloadLength);
139                 }
140 
141                 expectedChecksum = in.readInt();
142             } catch (EOFException e) {
143                 throw new CatalogFormatException("заголовок файла 
    обрезан", e);
144             }
145 
146             byte[] payload = in.readNBytes(payloadLength);
147             if (payload.length != payloadLength) {
148                 throw new CatalogFormatException("payload обрезан: 
    ожидалось "
149                         + payloadLength + " байт, получено " + 
    payload.length);
150             }
151 
152             if (in.read() != -1) {
153                 throw new CatalogFormatException("лишние данные 
    после payload");
154             }
155 
156             CRC32 crc = new CRC32();
157             crc.update(payload);
158             if ((int) crc.getValue() != expectedChecksum) {
159                 throw new CatalogFormatException("checksum не 
    совпал: данные повреждены");
160             }
161 
162             return parsePayload(version, payload);
163         }
164     }
165 
166     private static LoadedCatalog parsePayload(int version, byte[] 
    payload) throws IOException {
167         try (DataInputStream in = new DataInputStream(new 
    ByteArrayInputStream(payload))) {
168             int count;
169             try {
170                 count = in.readInt();
```

```
171             } catch (EOFException e) {
172                 throw new CatalogFormatException("payload обрезан:
    нет количества записей", e);
173             }
174
175             if (count < 0 || count > MAX_RECORDS) {
176                 throw new CatalogFormatException("недопустимое
    количество записей: " + count);
177             }
178
179             List<Course> courses = new ArrayList<>(count);
180             for (int i = 0; i < count; i++) {
181                 try {
182                     String name = in.readUTF();
183                     int hours = in.readInt();
184                     int credits = version == VERSION_1 ?
    creditsFromHours(hours) : in.readInt();
185                     validateCourse(name, hours, credits, i + 1);
186                     courses.add(new Course(name, hours, credits));
187                 } catch (EOFException e) {
188                     throw new CatalogFormatException("payload
    обрезан в записи #" + (i + 1), e);
189                 } catch (UTFDataFormatException e) {
190                     throw new CatalogFormatException("повреждённая
    строка в записи #" + (i + 1), e);
191                 }
192             }
193
194             if (in.available() != 0) {
195                 throw new CatalogFormatException("после записей
    остались лишние байты");
196             }
197
198             return new LoadedCatalog(version, List.copyOf(courses));
199         }
200     }
201
202     private static int creditsFromHours(int hours) {
203         return Math.max(1, hours / 36);
204     }
205
206     private static void validateCourse(String name, int hours, int
    credits, int index)
207             throws CatalogFormatException {
208         if (name == null || name.isBlank()) {
209             throw new CatalogFormatException("пустое название в
    записи #" + index);
```

```
210         }
211         if (hours <= 0) {
212             throw new CatalogFormatException("неверные часы в записи
    #" + index);
213         }
214         if (credits <= 0) {
215             throw new CatalogFormatException("неверные кредиты в
    записи #" + index);
216         }
217     }
218
219     private static void writeRawVersion(Path path, int version)
    throws IOException {
220         byte[] payload = new byte[] {0, 0, 0, 0};
221         CRC32 crc = new CRC32();
222         crc.update(payload);
223
224         try (DataOutputStream out = new DataOutputStream(
225                 new
    BufferedOutputStream(Files.newOutputStream(path)))) {
226             out.writeInt(MAGIC);
227             out.writeInt(version);
228             out.writeInt(payload.length);
229             out.writeInt((int) crc.getValue());
230             out.write(payload);
231         }
232     }
233
234     private static boolean expectRejected(Path path, String label) {
235         try {
236             LoadedCatalog catalog = readCatalog(path);
237             System.out.println("ОШИБКА: " + label + " принят: " +
    catalog);
238             return false;
239         } catch (UnsupportedVersionException e) {
240             System.out.println(label + " отклонён: " +
    e.getMessage());
241             return true;
242         } catch (CatalogFormatException e) {
243             System.out.println(label + " отклонён: " +
    e.getMessage());
244             return true;
245         } catch (IOException e) {
246             System.out.println(label + " отклонён (I/O): " +
    e.getMessage());
247             return true;
248         }
249     }
```

```
250
251     private static void deleteRecursively(Path dir) {
252         if (dir == null) {
253             return;
254         }
255         try (Stream<Path> paths = Files.walk(dir)) {
256             paths.sorted(Comparator.reverseOrder()).forEach(path ->
    {
257                 try {
258                     Files.deleteIfExists(path);
259                 } catch (IOException ignored) {
260                 }
261             });
262         } catch (IOException ignored) {
263         }
264     }
265
266     private record Course(String name, int hours, int credits) {
267     }
268
269     private record LoadedCatalog(int sourceVersion, List<Course>
    courses) {
270     }
271
272     private static class CatalogFormatException extends IOException
    {
273         private static final long serialVersionUID = 1L;
274
275         private CatalogFormatException(String message) {
276             super(message);
277         }
278
279         private CatalogFormatException(String message, Throwable
    cause) {
280             super(message, cause);
281         }
282     }
283
284     private static final class UnsupportedVersionException extends
    CatalogFormatException {
285         private static final long serialVersionUID = 1L;
286
287         private UnsupportedVersionException(int version) {
288             super("неизвестная версия формата: " + version);
289         }
290     }
```

```
291
292       private static final class Publisher {
293           private List<Course> catalog;
294
295           private Publisher(List<Course> initial) {
296               this.catalog = List.copyOf(initial);
297           }
298
299           private void publish(List<Course> newCatalog) {
300               this.catalog = List.copyOf(newCatalog);
301           }
302
303           private List<Course> catalog() {
304               return catalog;
305           }
306       }
307   }
```